

PROJECT REPORT

Project Name:

Inverted Pendulum Simulation and Control Using Python

Team Members:

Name	Scholar ID
Agrima Goel	2315022
Neha Evane	2315073
Sweta Maurya	2315080
Tanvi Sharma	2315095

Abstract

This report describes the simulation, control, and analysis of an inverted pendulum system using Python. The goal is to swing the pendulum from the downward position to upright and maintain its balance for at least two seconds. Energy-based swing-up control and Linear Quadratic Regulator (LQR) stabilisation are implemented using Python libraries. The project provides insights into control system design and demonstrates practical visualisation of system behaviour.

1. Introduction

The inverted pendulum exemplifies a classic engineering challenge due to its unstable nature. Consisting of a cart and a vertical rod, this system requires careful control to bring the pendulum upright and keep it balanced. This project uses Python for modelling, simulation, and control design, with plots and animation for result visualisation. Such techniques have broad utility in modern automation, robotics, and control engineering.

2. System Description

Parameters:

- Pendulum mass (m): 0.2 kg
- Cart mass (M): 0.5 kg
- Pendulum length (L): 0.3 m
- Gravity (g): 9.81 m/s²

State Variables:

- Cart position (x)
 - Cart velocity ($\dot{x}$)
 - Pendulum angle (θ), from vertical down
 - Pendulum angular velocity ($\dot{\theta}$)
-

3. Mathematical Modelling

The non-linear equations describe how the cart and pendulum interact dynamically:

$$\begin{aligned}\ddot{x} &= \frac{1}{D} [-m^2 L^2 g \cos \theta \sin \theta + mL^2 (mL \dot{\theta}^2 \sin \theta) + mL^2 u] \\ \ddot{\theta} &= \frac{1}{D} [(m + M)mgL \sin \theta - mL \cos \theta (mL \dot{\theta}^2 \sin \theta) - mL \cos \theta u]\end{aligned}$$

where

$$D = mL^2(M + m(1 - \cos^2 \theta))$$

and u is the control force applied to the cart.

4. Control Strategy

4.1 Energy-Based Swing-Up

The pendulum starts downward. The cart moves strategically to add energy, enabling the pendulum to rotate upward through 360 degrees.

4.2 LQR Stabilisation

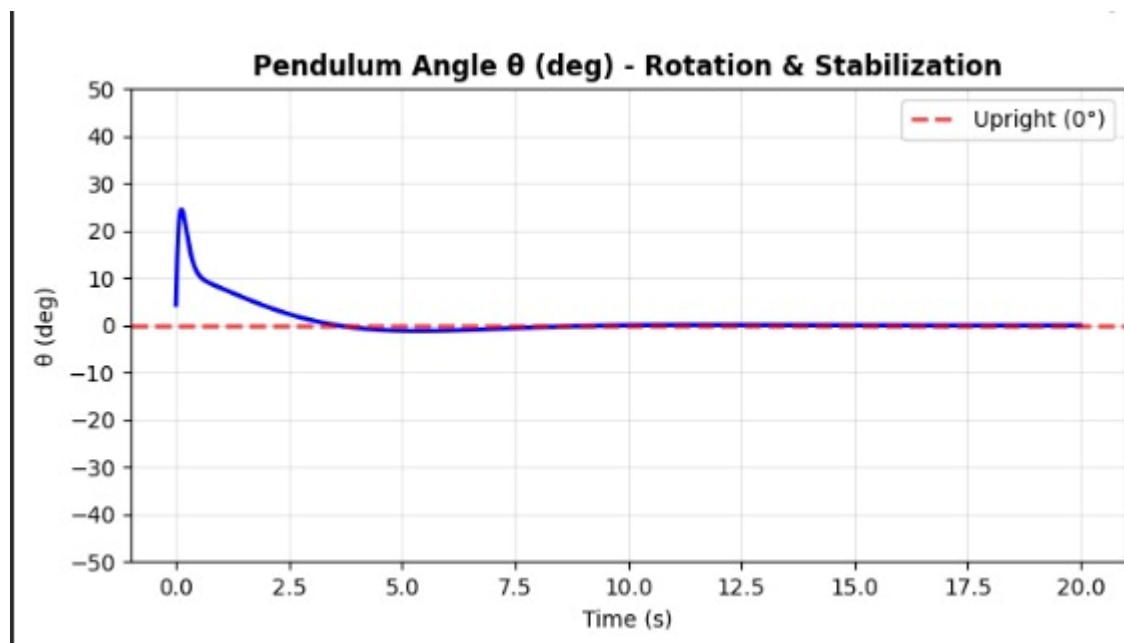
Once upright, LQR control maintains balance. LQR uses feedback from all state variables to calculate control forces, providing robust and optimal stabilisation. It is more effective than PID when controlling systems with multiple interacting variables.

5. Python Simulation Approach

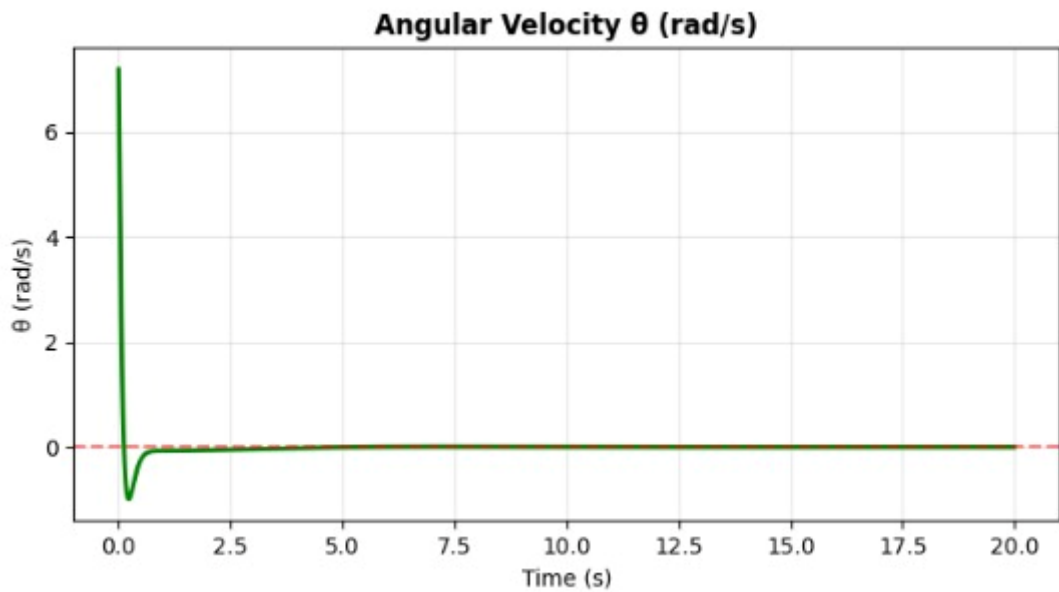
- **Tools:** numpy (math), matplotlib (plots, animation), scipy (integration, control)
 - **Integration:** Typically uses `scipy.integrate.solve_ivp` for time-stepping.
 - **Control Switching:** Swing-up strategy activates until near the upright position, then LQR begins for stabilisation.
 - **Outputs:** Graphs of system variables and animated visualisation.
-

6. Results

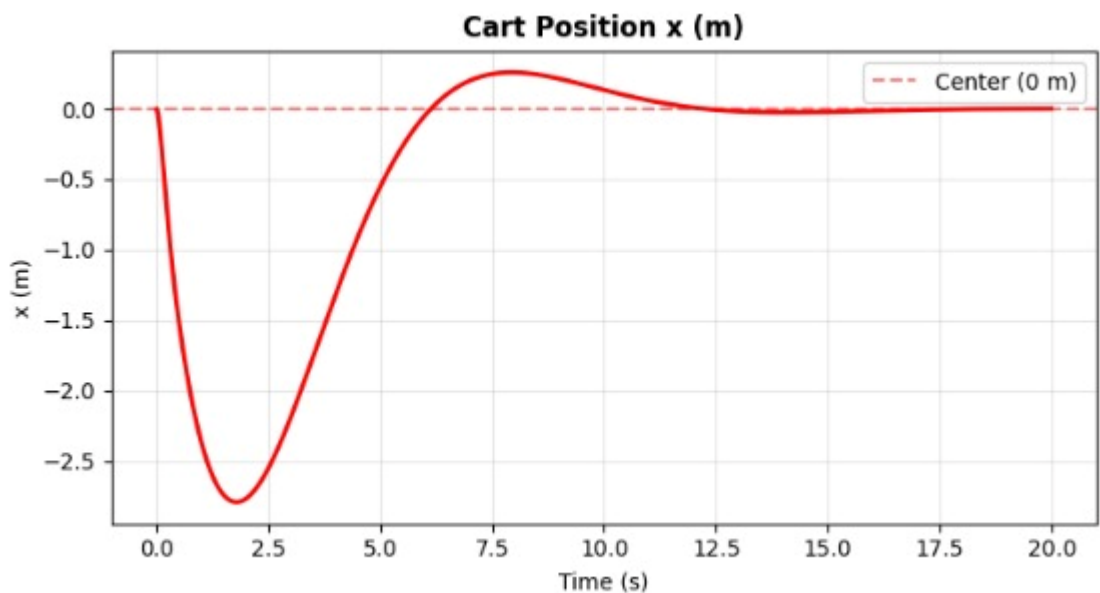
- **Waveform Plots:** Cart position, velocity, pendulum angle, and angular velocity waveforms display system swing-up and successful stabilisation for two seconds or more.
-



This graph shows the pendulum's angle starting with a large disturbance, then quickly decreasing as the controller stabilizes it. The oscillations gradually die out, and the pendulum settles near the upright position (0°), indicating successful rotation and stabilization by the control system.

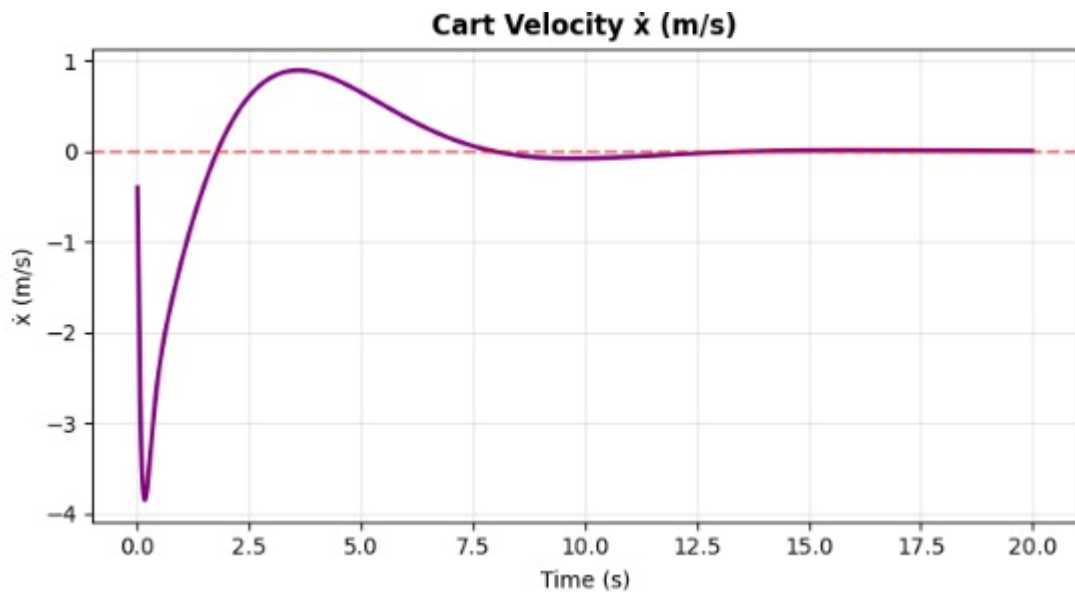


The graph shows the angular velocity of the inverted pendulum, which initially spikes due to the system's corrective action and then rapidly settles near zero. This quick damping indicates that the controller effectively suppresses rotational motion. Overall, the system reaches a stable upright condition with minimal steady-state velocity.

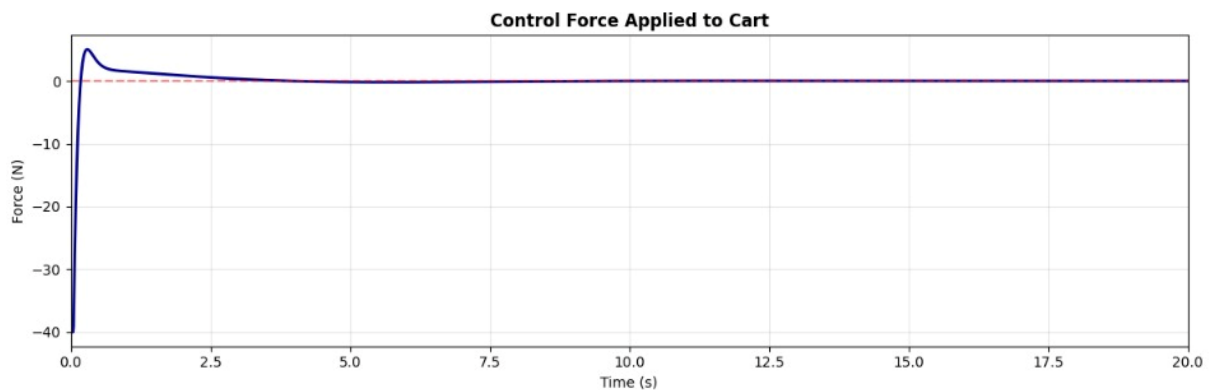


The cart position initially moves away from the centre as the controller reacts to balance the inverted pendulum, then gradually returns toward the 0 m reference. The overshoot and subsequent settling indicate the system compensating for the

pendulum's instability. Overall, the cart stabilises near the centre once the pendulum is balanced.



The plot shows the cart's velocity over time, starting with a sharp negative dip, then rising to a positive peak before gradually settling toward zero. This indicates an initial rapid backward motion, followed by forward movement and a damped return to rest.



The graph shows the control force applied to the cart, which begins with a large corrective impulse and then quickly reduces in magnitude. After the initial stabilisation effort, the force smoothly decays and settles near zero, indicating the controller has brought the system to steady state.

- **Animation:** Pendulum visually swings up and remains vertical, confirming effective control.

Animation link:

https://colab.research.google.com/drive/1cOEa7mqP5hrQYqq7GEamgFbxmEWIgV8h#scrollTo=GU_a-Jzv1Cks

7. Discussion

The combined swing-up and LQR approach handles this nonlinear, unstable system effectively. Energy-based control ensures sufficient momentum for rotation, while LQR delivers stable, responsive balancing using system feedback.

8. Conclusion

This simulation validates the suitability of Python for engineering control analysis and visualisation. The inverted pendulum is successfully swung up and stabilised using energy-based and LQR controllers. The approach can be generalised for similar unstable systems.

9. Future Outcomes and Applications

- **Advanced Control Research:** Techniques used for the inverted pendulum are foundational for designing advanced control algorithms in robotics (e.g., walking robots, bipedal robots).
- **Practical Systems:** Applications include self-balancing vehicles (like Segways, intelligent wheelchairs), rocket and missile stabilisation, and robotic arms.
- **Education:** This simulation is useful for teaching control theory, demonstrating concepts of stability, feedback, and system dynamics.
- **Software Development:** Python-based simulation frameworks can be extended for the development of more complex automated and intelligent systems.

11. References

- Ogata, K. "Modern Control Engineering." Prentice Hall.
- Documentation for NumPy, Matplotlib, and Scipy.

Selected Research Papers:

- Astrom, K. J., & Furuta, K. (2000). "Swinging up a Pendulum by Energy Control." *Automatica*, 36(2), 287-295.

- Das, T., & McReynolds, D. (2017). "Balancing an Inverted Pendulum Using Linear Quadratic Regulator (LQR) and Pole Placement." *International Journal of Computer Applications*, 163(4), 7-11.

Online resources:

- MathWorks documentation: Inverted Pendulum Example, MATLAB & Simulink.
- PythonControl library documentation: <https://python-control.readthedocs.io/en/latest/>